

Modul Praktik Minggu 6 — Laporan Agregat dan Ekspor CSV di Streamlit

Mata kuliah: Pemrograman Data Bisnis

Program Studi/Kelas: Administrasi Bisnis — AB-2B

Teknologi: Python, Streamlit, VS Code, SQLite, pandas

Alokasi: 2 pertemuan × 90 menit

Studi kasus: Laporan Penjualan Kedai Rasa (data simulasi)

Status: Outline ACC; modul siap ditinjau.

Modul ini meneruskan Minggu 5 (validasi, relasi `products` ↔ `sales`, dan `JOIN`). Fokusnya mengubah data transaksi menjadi **halaman laporan bisnis di Streamlit** yang dapat difilter, dibaca, ditelusuri, dan diunduh sebagai CSV.

Batas aman: semua data di modul ini adalah data usaha simulasi. Jangan memasukkan data pelanggan atau transaksi nyata. Buat folder dan database latihan baru; jangan menimpa `penjualan.db` (Minggu 3–4) atau `penjualan_m5.db` (Minggu 5).

A. Posisi modul dalam alur belajar

- **Minggu 3:** menyimpan, membaca, mengubah, dan menghapus transaksi (CRUD) di SQLite.
- **Minggu 4:** mencari, memfilter, dan membuat rekap sederhana dari satu tabel.
- **Minggu 5:** menjaga mutu input dan menghubungkan katalog `products` dengan catatan `sales` lewat `JOIN`.
- **Minggu 6:** membuat **halaman laporan** dari dua tabel berelasi. Ringkasan, laporan per produk/kategori/bulan, grafik, dan file CSV semuanya mengikuti filter yang sama.

Analogi: tabel `sales` ibarat **buku kas harian** yang ditulis baris demi baris. Pemilik usaha jarang membaca buku kas lembar per lembar. Yang dibutuhkan adalah **rekap**: berapa omzet bulan ini, produk apa yang paling laku, kategori mana yang paling besar. Minggu 6 membuat rekap itu otomatis di aplikasi, dan rekapnya tetap bisa ditelusuri kembali ke buku kas.

B. Capaian pembelajaran

Setelah menyelesaikan kedua pertemuan, mahasiswa diharapkan dapat:

1. membedakan data detail (baris transaksi) dan data ringkasan (hasil agregasi);
2. memasang `COUNT`, `SUM`, `AVG`, dan `MAX` sebagai fungsi Python, lalu menampilkannya dengan `st.metric`;
3. menghubungkan filter tanggal dan kategori ke query memakai parameter `?`, sehingga semua angka berubah bersama;
4. membuat laporan per produk, per kategori, dan per bulan dengan `JOIN`, `GROUP BY`, dan `ORDER BY`, serta menyaring kelompok dengan `HAVING`;
5. menyusun laporan dalam `st.tabs`, memformat kolom rupiah, dan menangani kondisi tanpa data; dan
6. menyediakan tombol unduh CSV yang isinya sama dengan tabel di layar, lalu mencocokkan angka antarlaporan.

C. Hasil akhir latihan

Sebuah halaman **Laporan Penjualan Kedai Rasa** di Streamlit yang:

- mengisi sendiri 6 produk dan 30 transaksi simulasi (Agustus–September 2026) saat pertama kali dijalankan;
- memiliki sidebar filter: periode tanggal, kategori, dan omzet minimum per produk;
- menampilkan empat kartu ringkasan: omzet, jumlah transaksi, rata-rata per transaksi, dan transaksi terbesar;
- memiliki tab **Per Produk**, **Per Kategori**, **Per Bulan**, dan **Detail Transaksi**, masing-masing dengan tabel dan tombol **Unduh CSV**;
- menampilkan grafik batang/garis sebagai pelengkap tabel; dan
- memiliki panel **Cek kecocokan angka** untuk membuktikan bahwa semua total sama.

Berkas latihan: folder `pdb-minggu-06` berisi `app.py` dan `penjualan_m6.db`. Database dibuat otomatis.

D. Istilah kunci

Istilah	Makna sederhana	Contoh di modul
Data detail	Satu baris untuk satu kejadian/transaksi.	Tab Detail Transaksi .
Data ringkasan	Hasil pengolahan banyak baris menjadi sedikit angka.	Kartu Omzet Rp1.489.000.
Fungsi agregasi	Fungsi SQL yang meringkas banyak baris: <code>COUNT</code> , <code>SUM</code> , <code>AVG</code> , <code>MIN</code> , <code>MAX</code> .	<code>SUM(sales.total)</code> menghasilkan omzet.
<code>GROUP BY</code>	Mengelompokkan baris yang nilainya sama, lalu menghitung agregasi per kelompok.	Omzet per produk.
<code>HAVING</code>	Menyaring kelompok setelah <code>GROUP BY</code> .	Hanya produk dengan omzet $\geq$ Rp200.000.
<code>WHERE</code>	Menyaring baris sebelum dikelompokkan.	Hanya transaksi bulan September.
<code>COALESCE</code>	Mengganti <code>NULL</code> dengan nilai cadangan.	<code>COALESCE(SUM(total), 0)</code> menghasilkan 0 bila tidak ada data.
Parameter <code>?</code>	Tempat nilai input di query; nilainya dikirim terpisah dan aman.	<code>BETWEEN ? AND ?</code> .
DataFrame	Tabel di Python (pustaka pandas) yang mudah ditampilkan dan diekspor.	Hasil <code>pd.read_sql_query</code> .
CSV	File teks berisi tabel; kolom dipisahkan tanda tertentu, bisa dibuka di Excel/Sheets.	<code>laporan-produk_2026-08-03_2026-09-30.csv</code> .
Rekonsiliasi	Mencocokkan angka dari beberapa laporan agar terbukti konsisten.	Total kategori = total bulan = total detail.

E. Persiapan komputer dan folder latihan

1. Buat folder baru

1. Buka VS Code.
2. Pilih **File** → **Open Folder...** lalu buat/pilih folder bernama `pdb-minggu-06`.
3. Jangan membuka atau menimpa folder Minggu 3–5. Jangan menyalin `penjualan_m5.db` ke folder ini; Minggu 6 memakai database sendiri.
4. Buka menu **Terminal** → **New Terminal**. Pastikan terminal berada di dalam `pdb-minggu-06`.

2. Siapkan Python, Streamlit, dan pandas

Pakai virtual environment kuliah yang sudah ada, atau buat yang baru di folder latihan:

```
python -m venv .venv
```

Aktifkan pada Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

Jika memakai Command Prompt:

```
.venv\Scripts\activate.bat
```

macOS/Linux:

```
source .venv/bin/activate
```

Setelah awalan `(.venv)` terlihat, pasang atau perbarui Streamlit. Pustaka **pandas** ikut terpasang bersama Streamlit, jadi tidak perlu dipasang terpisah.

```
python -m pip install --upgrade streamlit
python -m streamlit --version
python -c "import pandas; print(pandas.__version__)"
```

Modul ini diuji dengan **Streamlit 1.58** dan **pandas 3.0**. Beberapa opsi yang dipakai (`format="localized"` pada kolom tabel dan `sort=` pada grafik) belum tersedia di Streamlit versi lama. Jika muncul pesan *unexpected keyword argument*, jalankan lagi perintah `--upgrade` di atas.

3. Buat file aplikasi

Di Explorer, klik **New File**, beri nama `app.py`, lalu simpan. Kode setiap pertemuan adalah aplikasi lengkap. Salin **seluruh** blok ke `app.py`, jangan sepotong-sepotong.

Pertemuan 6.1 — Halaman Laporan dengan Filter dan Agregasi

Durasi: 90 menit

Pertanyaan pemantik: pemilik Kedai Rasa bertanya, "Berapa omzet minuman bulan September, dan berapa rata-rata nilai satu transaksi?" Bagaimana aplikasi menjawabnya langsung, tanpa menghitung baris satu per satu?

F. Alur belajar pertemuan 6.1

Waktu	Kegiatan
0–10 menit	Review <code>JOIN</code> Minggu 5. Siapkan folder <code>pdb-minggu-06</code> , salin kode 6.1, lalu jalankan. Amati tabel detail 30 transaksi.
10–30 menit	Kartu ringkasan: bahas <code>COUNT</code> , <code>SUM</code> , <code>AVG</code> , <code>MAX</code> , dan <code>COALESCE</code> . Cocokkan satu angka dengan hitung manual.
30–50 menit	Filter sidebar: rentang tanggal dan kategori dikirim ke query sebagai parameter <code>?</code> . Bahas bahaya menyambung teks input ke SQL.
50–75 menit	Laporan per produk: <code>GROUP BY</code> , <code>ORDER BY</code> , format kolom rupiah, dan slider omzet minimum sebagai <code>HAVING</code> .
75–85 menit	Uji di aplikasi: periode berisi data, periode kosong, kategori kosong, dan <code>HAVING</code> yang terlalu tinggi. Isi tabel uji.
85–90 menit	Rangkuman <code>WHERE</code> vs <code>HAVING</code> . Simpan checkpoint 6.1.

1. Data detail dan data ringkasan

Empat transaksi pertama di data simulasi:

Tanggal	Produk	Harga satuan	Jumlah	Total
2026-08-03	Kopi Susu	15.000	3	45.000
2026-08-03	Roti Cokelat	10.000	2	20.000
2026-08-05	Nasi Goreng	22.000	2	44.000
2026-08-07	Teh Lemon	12.000	4	48.000

Itu **data detail**. Jika ditanya "berapa omzet 3–7 Agustus?", kita menjumlahkan kolom total: 45.000 + 20.000 + 44.000 + 48.000 = **157.000**. Ada **4** transaksi, jadi rata-ratanya $157.000 \div 4 =$ **39.250**. Hasil seperti ini adalah **data ringkasan**.

Hitungan itu mudah untuk empat baris, tetapi tidak untuk ratusan baris. Fungsi agregasi SQL mengerjakannya:

Fungsi	Pertanyaan bisnis	Query
COUNT(*)	Berapa transaksi terjadi?	SELECT COUNT(*) FROM sales
SUM(kolom)	Berapa total omzet/unit?	SELECT SUM(total) FROM sales
AVG(kolom)	Berapa rata-rata nilai transaksi?	SELECT AVG(total) FROM sales
MAX(kolom) / MIN(kolom)	Berapa transaksi terbesar/terkecil?	SELECT MAX(total) FROM sales

Nanti angka manual di atas akan dicocokkan dengan aplikasi dengan cara memfilter periode 03/08/2026–07/08/2026.

2. Ketik dan simpan aplikasi lengkap Pertemuan 6.1

1. Klik tab `app.py`.
2. Pilih seluruh isi file, lalu ganti dengan kode berikut.
3. Periksa indentasi: isi fungsi dan isi `if/with` bertambah empat spasi.
4. Simpan dengan **Ctrl+S** (macOS: **Cmd+S**).

```
from contextlib import closing
from datetime import date
from pathlib import Path
import sqlite3

import pandas as pd
import streamlit as st

DB_PATH = Path(__file__).with_name("penjualan_m6.db")
SEED_PRODUCTS = [
    ("Kopi Susu", "Minuman", 15000),
    ("Teh Lemon", "Minuman", 12000),
    ("Es Cokelat", "Minuman", 18000),
    ("Roti Cokelat", "Makanan", 10000),
    ("Nasi Goreng", "Makanan", 22000),
    ("Pisang Goreng", "Camilan", 8000),
]

# (tanggal, nama produk, harga per unit saat transaksi, jumlah unit)
SEED_SALES = [
    ("2026-08-03", "Kopi Susu", 15000, 3),
    ("2026-08-03", "Roti Cokelat", 10000, 2),
    ("2026-08-05", "Nasi Goreng", 22000, 2),
    ("2026-08-07", "Teh Lemon", 12000, 4),
    ("2026-08-10", "Pisang Goreng", 8000, 5),
    ("2026-08-12", "Es Cokelat", 18000, 2),
    ("2026-08-14", "Kopi Susu", 15000, 5),
    ("2026-08-17", "Kopi Susu", 13000, 6),
    ("2026-08-17", "Pisang Goreng", 8000, 6),
    ("2026-08-19", "Nasi Goreng", 22000, 3),
    ("2026-08-21", "Roti Cokelat", 10000, 3),
    ("2026-08-24", "Teh Lemon", 12000, 2),
    ("2026-08-26", "Es Cokelat", 18000, 3),
    ("2026-08-28", "Kopi Susu", 15000, 4),
    ("2026-08-31", "Nasi Goreng", 22000, 1),
    ("2026-09-01", "Kopi Susu", 15000, 4),
    ("2026-09-02", "Pisang Goreng", 8000, 4),
    ("2026-09-04", "Nasi Goreng", 22000, 4),
    ("2026-09-07", "Teh Lemon", 12000, 5),
    ("2026-09-09", "Roti Cokelat", 10000, 4),
    ("2026-09-11", "Es Cokelat", 18000, 3),
    ("2026-09-14", "Kopi Susu", 15000, 6),
    ("2026-09-16", "Nasi Goreng", 22000, 2),
```

```

("2026-09-18", "Pisang Goreng", 8000, 3),
("2026-09-21", "Es Cokelat", 16000, 5),
("2026-09-23", "Teh Lemon", 12000, 3),
("2026-09-25", "Kopi Susu", 15000, 5),
("2026-09-28", "Roti Cokelat", 10000, 2),
("2026-09-30", "Nasi Goreng", 22000, 3),
("2026-09-30", "Kopi Susu", 15000, 2),
]
JOIN_SQL = "FROM sales JOIN products ON products.id = sales.product_id"

def get_connection():
    """Buka koneksi SQLite dan aktifkan pemeriksaan foreign key."""
    connection = sqlite3.connect(DB_PATH)
    connection.execute("PRAGMA foreign_keys = ON")
    return connection

def initialize_database():
    """Buat tabel, isi katalog, dan isi transaksi simulasi jika masih kosong."""
    with closing(get_connection()) as connection:
        connection.executescript(
            """
            CREATE TABLE IF NOT EXISTS products (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL UNIQUE,
                category TEXT NOT NULL,
                catalog_price INTEGER NOT NULL CHECK (catalog_price > 0)
            );

            CREATE TABLE IF NOT EXISTS sales (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                sale_date TEXT NOT NULL,
                product_id INTEGER NOT NULL,
                unit_price INTEGER NOT NULL CHECK (unit_price > 0),
                quantity INTEGER NOT NULL CHECK (quantity > 0),
                total INTEGER NOT NULL CHECK (total = unit_price * quantity),
                FOREIGN KEY (product_id) REFERENCES products(id)
            );
            """
        )
        connection.executemany(
            """
            INSERT OR IGNORE INTO products (name, category, catalog_price)
            VALUES (?, ?, ?)
            """
        )
        SEED_PRODUCTS,
    )
    sales_count = connection.execute("SELECT COUNT(*) FROM sales").fetchone()[0]
    if sales_count == 0:
        product_ids = dict(
            connection.execute("SELECT name, id FROM products").fetchall()
        )
        connection.executemany(
            """
            INSERT INTO sales
            (sale_date, product_id, unit_price, quantity, total)
            VALUES (?, ?, ?, ?, ?)
            """
        )
        [
            (sale_date, product_ids[name], price, quantity, price * quantity)
            for sale_date, name, price, quantity in SEED_SALES
        ],
    )
    connection.commit()

def run_query(sql, params=()):
    """Jalankan query SELECT dan kembalikan hasilnya sebagai tabel DataFrame."""

```

```

with closing(get_connection()) as connection:
    return pd.read_sql_query(sql, connection, params=list(params))

def get_categories():
    """Ambil daftar kategori untuk pilihan filter."""
    result = run_query("SELECT DISTINCT category FROM products ORDER BY category")
    return result["category"].tolist()

def get_date_bounds():
    """Ambil tanggal transaksi pertama dan terakhir sebagai periode bawaan."""
    result = run_query(
        "SELECT MIN(sale_date) AS first_date, MAX(sale_date) AS last_date FROM sales"
    )
    first_date, last_date = result.iloc[0]
    if first_date is None:
        return date.today(), date.today()
    return date.fromisoformat(first_date), date.fromisoformat(last_date)

def build_filter(start_date, end_date, categories):
    """Susun bagian WHERE dan nilai parameternya dari pilihan filter."""
    placeholders = ", ".join("{}" for _ in categories)
    where_sql = (
        "WHERE sales.sale_date BETWEEN ? AND ? "
        f"AND products.category IN ({placeholders})"
    )
    params = [start_date.isoformat(), end_date.isoformat(), *categories]
    return where_sql, params

def get_summary(where_sql, params):
    """Hitung angka ringkasan untuk semua transaksi yang lolos filter."""
    result = run_query(
        f"""
        SELECT
            COUNT(*) AS jumlah_transaksi,
            COALESCE(SUM(sales.quantity), 0) AS unit_terjual,
            COALESCE(SUM(sales.total), 0) AS omzet,
            COALESCE(AVG(sales.total), 0) AS rata_rata,
            COALESCE(MAX(sales.total), 0) AS transaksi_terbesar
        {JOIN_SQL}
        {where_sql}
        """,
        params,
    )
    return result.iloc[0]

def get_report_by_product(where_sql, params, min_revenue):
    """Laporan per produk; HAVING menyaring produk dengan omzet kecil."""
    return run_query(
        f"""
        SELECT
            products.name AS produk,
            products.category AS kategori,
            COUNT(*) AS jumlah_transaksi,
            SUM(sales.quantity) AS unit_terjual,
            SUM(sales.total) AS omzet
        {JOIN_SQL}
        {where_sql}
        GROUP BY products.id, products.name, products.category
        HAVING SUM(sales.total) >= ?
        ORDER BY omzet DESC, produk
        """,
        [*params, min_revenue],
    )

```

```

def get_sales_detail(where_sql, params):
    """Data detail (baris per transaksi) untuk menelusuri angka laporan."""
    return run_query(
        f"""
        SELECT
            sales.id AS nomor,
            sales.sale_date AS tanggal,
            products.name AS produk,
            products.category AS kategori,
            sales.unit_price AS harga_satuan,
            sales.quantity AS jumlah,
            sales.total AS total
        {JOIN_SQL}
        {where_sql}
        ORDER BY sales.sale_date, sales.id
        """,
        params,
    )

def format_rupiah(value):
    """Ubah angka menjadi teks rupiah, misalnya 1500000 -> Rp1.500.000."""
    return f"Rp{round(value):,}".replace(",", ".")

COLUMN_CONFIG = {
    "nomor": st.column_config.NumberColumn("No."),
    "tanggal": st.column_config.TextColumn("Tanggal"),
    "produk": st.column_config.TextColumn("Produk"),
    "kategori": st.column_config.TextColumn("Kategori"),
    "jumlah_transaksi": st.column_config.NumberColumn("Jumlah transaksi"),
    "unit_terjual": st.column_config.NumberColumn("Unit terjual"),
    "jumlah": st.column_config.NumberColumn("Jumlah unit"),
    "harga_satuan": st.column_config.NumberColumn(
        "Harga satuan (Rp)", format="localized"
    ),
    "total": st.column_config.NumberColumn("Total (Rp)", format="localized"),
    "omzet": st.column_config.NumberColumn("Omzet (Rp)", format="localized"),
}

def main():
    st.set_page_config(page_title="Kedai Rasa – Minggu 6", layout="wide")
    st.title("Laporan Penjualan Kedai Rasa")
    st.caption("Latihan Minggu 6 tahap 6.1. Semua angka berasal dari data simulasi.")

    initialize_database()
    first_date, last_date = get_date_bounds()
    categories = get_categories()

    with st.sidebar:
        st.header("Filter laporan")
        period = st.date_input(
            "Periode",
            value=(first_date, last_date),
            format="DD/MM/YYYY",
        )
        selected_categories = st.multiselect(
            "Kategori", options=categories, default=categories
        )
        min_revenue = st.slider(
            "Omzet minimum per produk (Rp)",
            min_value=0,
            max_value=500000,
            value=0,
            step=25000,
        )

```

```

if len(period) != 2:
    st.info("Pilih tanggal awal dan tanggal akhir periode di sidebar.")
    st.stop()
if not selected_categories:
    st.warning("Pilih minimal satu kategori di sidebar.")
    st.stop()

start_date, end_date = period
where_sql, params = build_filter(start_date, end_date, selected_categories)
summary = get_summary(where_sql, params)

st.subheader(f"Ringkasan {start_date:%d/%m/%Y} - {end_date:%d/%m/%Y}")
col1, col2, col3, col4 = st.columns(4)
col1.metric("Omzet", format_rupiah(summary["omzet"]))
col2.metric("Jumlah transaksi", int(summary["jumlah_transaksi"]))
col3.metric("Rata-rata per transaksi", format_rupiah(summary["rata_rata"]))
col4.metric("Transaksi terbesar", format_rupiah(summary["transaksi_terbesar"]))

if summary["jumlah_transaksi"] == 0:
    st.info("Tidak ada transaksi pada periode dan kategori ini.")
    st.stop()

st.subheader("Laporan per produk")
st.caption(f"Produk dengan omzet minimal {format_rupiah(min_revenue)}.")
by_product = get_report_by_product(where_sql, params, min_revenue)
if by_product.empty:
    st.info("Tidak ada produk yang mencapai omzet minimum tersebut.")
else:
    st.dataframe(
        by_product,
        column_config=COLUMN_CONFIG,
        hide_index=True,
        width="stretch",
    )

st.subheader("Detail transaksi sesuai filter")
st.dataframe(
    get_sales_detail(where_sql, params),
    column_config=COLUMN_CONFIG,
    hide_index=True,
    width="stretch",
)

st.info(
    "Checkpoint 6.1: ubah filter di sidebar, lalu pastikan kartu ringkasan, "
    "laporan per produk, dan detail transaksi berubah bersama."
)

if __name__ == "__main__":
    main()

```

3. Jalankan dan amati hasil

Jalankan di terminal:

```
python -m streamlit run app.py
```

Browser terbuka dan menampilkan halaman **Laporan Penjualan Kedai Rasa**. Periksa:

1. File `penjualan_m6.db` muncul di Explorer VS Code. Database ini dibuat dan diisi data simulasi oleh `initialize_database()`.
2. Sidebar kiri berisi **Periode**, **Kategori**, dan **Omzet minimum per produk**.
3. Empat kartu ringkasan menunjukkan **Rp1.489.000**, **30**, **Rp49.633**, dan **Rp90.000**.

4. Tabel **Laporan per produk** diurutkan dari omzet terbesar (Kopi Susu di atas).
5. Tabel **Detail transaksi** berisi 30 baris.

Jika angkanya berbeda, kemungkinan data simulasi pernah diubah. Tutup Streamlit (**Ctrl+C** di terminal), hapus `penjualan_m6.db`, lalu jalankan lagi.

4. Bedah kode bagian demi bagian

a. Data simulasi otomatis

`SEED_SALES` berisi 30 transaksi contoh. Di `initialize_database()`, transaksi hanya diisi jika tabel `sales` masih kosong:

```
sales_count = connection.execute("SELECT COUNT(*) FROM sales").fetchone()[0]
if sales_count == 0:
    ...
```

Dengan begitu, data tidak ikut bertambah setiap kali Streamlit menjalankan ulang skrip. Skema tabel **sama persis** dengan Minggu 5, termasuk `CHECK` dan foreign key. Data simulasi tetap melewati aturan yang sama.

b. Hasil query langsung menjadi DataFrame

```
def run_query(sql, params=()):
    with closing(get_connection()) as connection:
        return pd.read_sql_query(sql, connection, params=list(params))
```

`pd.read_sql_query` menjalankan query dan mengembalikan tabel **DataFrame** lengkap dengan nama kolom. DataFrame bisa langsung diberikan ke `st.dataframe`, `st.bar_chart`, dan pada pertemuan 6.2 diubah menjadi CSV. Semua fungsi laporan memakai `run_query` agar cara mengambil data seragam.

c. Filter menjadi `WHERE` dengan parameter

```
def build_filter(start_date, end_date, categories):
    placeholders = ", ".join("?" for _ in categories)
    where_sql = (
        "WHERE sales.sale_date BETWEEN ? AND ? "
        f"AND products.category IN ({placeholders})"
    )
    params = [start_date.isoformat(), end_date.isoformat(), *categories]
    return where_sql, params
```

Jika memilih periode 01/09/2026–30/09/2026 dan kategori Minuman serta Makanan, hasilnya:

```
WHERE sales.sale_date BETWEEN ? AND ? AND products.category IN (?, ?)
params = ['2026-09-01', '2026-09-30', 'Minuman', 'Makanan']
```

Perhatikan bahwa f-string hanya menyusun **jumlah tanda ?**. Nilai pilihan pengguna tidak pernah ditempel ke teks SQL; nilainya dikirim lewat `params`. Cara ini mencegah *SQL injection* dan masalah tanda kutip. **Jangan** menulis `f"... category = '{kategori}'"`.

`BETWEEN` bisa dipakai pada tanggal karena tanggal disimpan sebagai teks berformat `YYYY-MM-DD`. Teks dengan format itu urutannya sama dengan urutan kalender.

Fungsi `build_filter` dipanggil **sekali**, lalu `where_sql` dan `params` yang sama dipakai oleh semua laporan. Itulah sebabnya mengubah filter membuat kartu ringkasan, laporan per produk, dan detail berubah bersama.

d. Kartu ringkasan dan `COALESCE`

```
SELECT
  COUNT(*) AS jumlah_transaksi,
  COALESCE(SUM(sales.quantity), 0) AS unit_terjual,
  COALESCE(SUM(sales.total), 0) AS omzet,
  COALESCE(AVG(sales.total), 0) AS rata_rata,
  COALESCE(MAX(sales.total), 0) AS transaksi_terbesar
FROM sales JOIN products ON products.id = sales.product_id
WHERE ...
```

Jika tidak ada baris yang lolos filter, `COUNT(*)` menghasilkan 0, tetapi `SUM`, `AVG`, dan `MAX` menghasilkan `NULL` (kosong). `COALESCE(..., 0)` menggantinya dengan 0 agar kartu tetap menampilkan `Rp0`, bukan error.

`COUNT(*)` menghitung **baris**. `COUNT(kolom)` hanya menghitung baris yang kolom tersebut tidak `NULL`. Di latihan ini kolom `sales` tidak boleh kosong (`NOT NULL`), jadi hasil keduanya sama.

Di Streamlit, keempat angka ditata dalam empat kolom:

```
col1, col2, col3, col4 = st.columns(4)
col1.metric("Omzet", format_rupiah(summary["omzet"]))
```

`format_rupiah` mengubah `1489000` menjadi `Rp1.489.000` (titik sebagai pemisah ribuan).

e. `GROUP BY`, `ORDER BY`, dan `HAVING`

```
SELECT products.name AS produk, products.category AS kategori,
  COUNT(*) AS jumlah_transaksi, SUM(sales.quantity) AS unit_terjual,
  SUM(sales.total) AS omzet
FROM sales JOIN products ON products.id = sales.product_id
WHERE ...
GROUP BY products.id, products.name, products.category
HAVING SUM(sales.total) >= ?
ORDER BY omzet DESC, produk
```

Urutan kerja database:

1. `FROM ... JOIN` menyatukan transaksi dengan nama dan kategori produk.
2. `WHERE` membuang **baris** di luar periode/kategori.
3. `GROUP BY` mengumpulkan baris yang produknya sama menjadi satu kelompok.
4. `COUNT / SUM` dihitung **per kelompok**.
5. `HAVING` membuang **kelompok** yang omzetnya di bawah batas slider.
6. `ORDER BY omzet DESC` mengurutkan dari omzet terbesar; jika sama,urut nama produk.

	WHERE	HAVING
Yang disaring	Baris transaksi	Kelompok hasil <code>GROUP BY</code>
Waktu bekerja	Sebelum pengelompokan	Sesudah agregasi
Boleh memakai <code>SUM(...)</code> ?	Tidak	Ya

	WHERE	HAVING
Contoh di aplikasi	Periode dan kategori	Omzet minimum per produk

f. Widget Streamlit yang baru

Widget	Kegunaan di aplikasi	Catatan
<code>with st.sidebar:</code>	Menaruh filter di panel kiri.	Halaman utama lebih lega untuk laporan.
<code>st.date_input(value=(awal, akhir))</code>	Memilih rentang tanggal.	Saat pengguna baru memilih satu tanggal, nilainya hanya berisi satu tanggal. Aplikasi menunggu dengan <code>st.info</code> lalu <code>st.stop()</code> .
<code>st.multiselect</code>	Memilih satu atau beberapa kategori.	Jika semua pilihan dihapus, aplikasi berhenti dengan peringatan.
<code>st.slider</code>	Batas omzet minimum untuk HAVING .	Nilai 0 berarti semua produk tampil.
<code>st.columns + st.metric</code>	Kartu angka ringkasan.	Menampilkan teks yang sudah diformat rupiah.
<code>st.column_config</code>	Judul kolom dalam bahasa Indonesia dan format angka.	<code>format="localized"</code> menampilkan pemisah ribuan sesuai bahasa browser.
<code>st.stop()</code>	Menghentikan sisa skrip pada putaran ini.	Dipakai ketika filter belum lengkap atau data kosong.

5. Angka acuan untuk dicocokkan

Ubah filter di sidebar, lalu cocokkan kartu dan tabel dengan angka berikut.

Filter	Transaksi	Omzet	Rata-rata	Terbesar
03/08/2026–30/09/2026, semua kategori	30	Rp1.489.000	Rp49.633	Rp90.000
03/08/2026–07/08/2026, semua kategori	4	Rp157.000	Rp39.250	Rp48.000
01/08/2026–31/08/2026, semua kategori	15	Rp690.000	Rp46.000	Rp78.000
01/09/2026–30/09/2026, semua kategori	15	Rp799.000	Rp53.267	Rp90.000
01/09/2026–30/09/2026, hanya Minuman	8	Rp485.000	Rp60.625	Rp90.000

Laporan per produk untuk seluruh periode (slider = 0):

Produk	Kategori	Transaksi	Unit	Omzet
Kopi Susu	Minuman	8	35	513.000
Nasi Goreng	Makanan	6	15	330.000
Es Cokelat	Minuman	4	13	224.000

Produk	Kategori	Transaksi	Unit	Omzet
Teh Lemon	Minuman	4	14	168.000
Pisang Goreng	Camilan	4	18	144.000
Roti Cokelat	Makanan	4	11	110.000

Pisang Goreng terjual **18 unit**, lebih banyak daripada Nasi Goreng (15 unit), tetapi omzetnya jauh lebih kecil. "Terlaris" perlu dijelaskan: terlaris menurut **unit** atau menurut **omzet**? Laporan yang baik menyebutkan dasar urutannya.

6. Uji di aplikasi

Catat hasilnya di `catatan-pengujian.txt`.

Kasus	Tindakan	Hasil yang diharapkan
Periode berisi data	Periode 01/09/2026–30/09/2026	Kartu sesuai angka acuan; detail 15 baris.
Cocok dengan hitung manual	Periode 03/08/2026–07/08/2026	Omzet Rp157.000, 4 transaksi, rata-rata Rp39.250.
Periode tanpa transaksi	Periode 01/10/2026–31/10/2026	Kartu menampilkan <code>Rp0</code> dan 0; muncul info "Tidak ada transaksi"; tidak error.
Rentang belum lengkap	Klik satu tanggal saja di kalender	Info "Pilih tanggal awal dan tanggal akhir"; tidak error.
Kategori kosong	Hapus semua kategori	Peringatan "Pilih minimal satu kategori"; tidak error.
<code>HAVING</code> menyaring	Slider ke Rp200.000 (seluruh periode)	Hanya Kopi Susu, Nasi Goreng, dan Es Cokelat.
<code>HAVING</code> terlalu tinggi	Slider ke Rp500.000 lalu periode September saja	Muncul info "Tidak ada produk yang mencapai omzet minimum"; kartu ringkasan tetap tampil.

Perhatikan kasus terakhir: `HAVING` hanya memengaruhi laporan per produk. Kartu ringkasan tidak memakai `HAVING`, jadi angkanya tidak berubah.

7. Checkpoint pertemuan 6.1

- [] `penjualan_m6.db` dibuat di folder `pdb-minggu-06`, bukan di folder minggu lain.
- [] Empat kartu ringkasan cocok dengan angka acuan.
- [] Filter periode dan kategori mengubah kartu, laporan produk, dan detail bersama-sama.
- [] Nilai filter dikirim lewat parameter `?`, bukan ditempel ke teks SQL.
- [] Slider omzet minimum bekerja sebagai `HAVING`.
- [] Periode kosong dan kategori kosong tidak menimbulkan error.

Pertemuan 6.2 — Laporan Bertab, Grafik, dan Ekspor CSV

Durasi: 90 menit

Pertanyaan pemantik: laporan sudah tampil di aplikasi. Bagaimana pemilik usaha membagikannya ke akuntan atau rekan yang tidak membuka aplikasi, dan bagaimana mereka yakin angkanya benar?

G. Alur belajar pertemuan 6.2

Waktu	Kegiatan
0–10 menit	Review tahap 6.1. Rencanakan susunan halaman: ringkasan di atas, laporan di dalam tab.
10–30 menit	Ganti <code>app.py</code> dengan kode 6.2. Bahas laporan per kategori, per bulan (<code>strftime</code>), dan <code>st.tabs</code> .
30–45 menit	Grafik pelengkap: <code>st.bar_chart</code> dan <code>st.line_chart</code> . Tabel tetap menjadi acuan angka.
45–70 menit	Ekspor CSV: <code>to_csv</code> dan <code>st.download_button</code> . Unduh dan buka file di Excel/Sheets.
70–85 menit	Rekonsiliasi: cocokkan total antar-tab, kartu, dan file CSV. Uji periode kosong.
85–90 menit	Refleksi dan jembatan ke UTS. Simpan <code>app.py</code> versi akhir.

1. Rencana halaman

Bagian halaman	Isi
Sidebar	Periode · Kategori · Omzet minimum per produk
Ringkasan	Omzet · Jumlah transaksi · Rata-rata · Transaksi terbesar
Tab	Per Produk · Per Kategori · Per Bulan · Detail Transaksi
Isi setiap tab	Tabel → tombol Unduh CSV → grafik
Bawah halaman	Panel Cek kecocokan angka

Semua tab memakai `where_sql` dan `params` yang sama dari sidebar. Mengubah satu filter berarti mengubah seluruh laporan.

2. Ganti `app.py` dengan aplikasi lengkap Pertemuan 6.2

1. Tutup tab browser lama atau biarkan terbuka.
2. Ganti seluruh isi `app.py` dengan kode berikut, lalu simpan.
3. Database `penjualan_m6.db` tetap dipakai dan tidak perlu dihapus.

```
from contextlib import closing
from datetime import date
from pathlib import Path
import sqlite3
```

```

import pandas as pd
import streamlit as st

DB_PATH = Path(__file__).with_name("penjualan_m6.db")
SEED_PRODUCTS = [
    ("Kopi Susu", "Minuman", 15000),
    ("Teh Lemon", "Minuman", 12000),
    ("Es Cokelat", "Minuman", 18000),
    ("Roti Cokelat", "Makanan", 10000),
    ("Nasi Goreng", "Makanan", 22000),
    ("Pisang Goreng", "Camilan", 8000),
]

# (tanggal, nama produk, harga per unit saat transaksi, jumlah unit)
SEED_SALES = [
    ("2026-08-03", "Kopi Susu", 15000, 3),
    ("2026-08-03", "Roti Cokelat", 10000, 2),
    ("2026-08-05", "Nasi Goreng", 22000, 2),
    ("2026-08-07", "Teh Lemon", 12000, 4),
    ("2026-08-10", "Pisang Goreng", 8000, 5),
    ("2026-08-12", "Es Cokelat", 18000, 2),
    ("2026-08-14", "Kopi Susu", 15000, 5),
    ("2026-08-17", "Kopi Susu", 13000, 6),
    ("2026-08-17", "Pisang Goreng", 8000, 6),
    ("2026-08-19", "Nasi Goreng", 22000, 3),
    ("2026-08-21", "Roti Cokelat", 10000, 3),
    ("2026-08-24", "Teh Lemon", 12000, 2),
    ("2026-08-26", "Es Cokelat", 18000, 3),
    ("2026-08-28", "Kopi Susu", 15000, 4),
    ("2026-08-31", "Nasi Goreng", 22000, 1),
    ("2026-09-01", "Kopi Susu", 15000, 4),
    ("2026-09-02", "Pisang Goreng", 8000, 4),
    ("2026-09-04", "Nasi Goreng", 22000, 4),
    ("2026-09-07", "Teh Lemon", 12000, 5),
    ("2026-09-09", "Roti Cokelat", 10000, 4),
    ("2026-09-11", "Es Cokelat", 18000, 3),
    ("2026-09-14", "Kopi Susu", 15000, 6),
    ("2026-09-16", "Nasi Goreng", 22000, 2),
    ("2026-09-18", "Pisang Goreng", 8000, 3),
    ("2026-09-21", "Es Cokelat", 16000, 5),
    ("2026-09-23", "Teh Lemon", 12000, 3),
    ("2026-09-25", "Kopi Susu", 15000, 5),
    ("2026-09-28", "Roti Cokelat", 10000, 2),
    ("2026-09-30", "Nasi Goreng", 22000, 3),
    ("2026-09-30", "Kopi Susu", 15000, 2),
]

JOIN_SQL = "FROM sales JOIN products ON products.id = sales.product_id"

def get_connection():
    """Buka koneksi SQLite dan aktifkan pemeriksaan foreign key."""
    connection = sqlite3.connect(DB_PATH)
    connection.execute("PRAGMA foreign_keys = ON")
    return connection

def initialize_database():
    """Buat tabel, isi katalog, dan isi transaksi simulasi jika masih kosong."""
    with closing(get_connection()) as connection:
        connection.executescript(
            """
            CREATE TABLE IF NOT EXISTS products (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL UNIQUE,
                category TEXT NOT NULL,
                catalog_price INTEGER NOT NULL CHECK (catalog_price > 0)
            );

            CREATE TABLE IF NOT EXISTS sales (
                id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        sale_date TEXT NOT NULL,
        product_id INTEGER NOT NULL,
        unit_price INTEGER NOT NULL CHECK (unit_price > 0),
        quantity INTEGER NOT NULL CHECK (quantity > 0),
        total INTEGER NOT NULL CHECK (total = unit_price * quantity),
        FOREIGN KEY (product_id) REFERENCES products(id)
    );
    """
)
connection.executemany(
    """
    INSERT OR IGNORE INTO products (name, category, catalog_price)
    VALUES (?, ?, ?)
    """,
    SEED_PRODUCTS,
)
sales_count = connection.execute("SELECT COUNT(*) FROM sales").fetchone()[0]
if sales_count == 0:
    product_ids = dict(
        connection.execute("SELECT name, id FROM products").fetchall()
    )
    connection.executemany(
        """
        INSERT INTO sales
            (sale_date, product_id, unit_price, quantity, total)
        VALUES (?, ?, ?, ?, ?)
        """,
        [
            (sale_date, product_ids[name], price, quantity, price * quantity)
            for sale_date, name, price, quantity in SEED_SALES
        ],
    )
    connection.commit()

def run_query(sql, params=()):
    """Jalankan query SELECT dan kembalikan hasilnya sebagai tabel DataFrame."""
    with closing(get_connection()) as connection:
        return pd.read_sql_query(sql, connection, params=list(params))

def get_categories():
    """Ambil daftar kategori untuk pilihan filter."""
    result = run_query("SELECT DISTINCT category FROM products ORDER BY category")
    return result["category"].tolist()

def get_date_bounds():
    """Ambil tanggal transaksi pertama dan terakhir sebagai periode bawaan."""
    result = run_query(
        "SELECT MIN(sale_date) AS first_date, MAX(sale_date) AS last_date FROM sales"
    )
    first_date, last_date = result.iloc[0]
    if first_date is None:
        return date.today(), date.today()
    return date.fromisoformat(first_date), date.fromisoformat(last_date)

def build_filter(start_date, end_date, categories):
    """Susun bagian WHERE dan nilai parameternya dari pilihan filter."""
    placeholders = ", ".join("?" for _ in categories)
    where_sql = (
        "WHERE sales.sale_date BETWEEN ? AND ? "
        f"AND products.category IN ({placeholders})"
    )
    params = [start_date.isoformat(), end_date.isoformat(), *categories]
    return where_sql, params

```

```

def get_summary(where_sql, params):
    """Hitung angka ringkasan untuk semua transaksi yang lolos filter."""
    result = run_query(
        f"""
        SELECT
            COUNT(*) AS jumlah_transaksi,
            COALESCE(SUM(sales.quantity), 0) AS unit_terjual,
            COALESCE(SUM(sales.total), 0) AS omzet,
            COALESCE(AVG(sales.total), 0) AS rata_rata,
            COALESCE(MAX(sales.total), 0) AS transaksi_terbesar
        {JOIN_SQL}
        {where_sql}
        """,
        params,
    )
    return result.iloc[0]

```

```

def get_report_by_product(where_sql, params, min_revenue):
    """Laporan per produk; HAVING menyaring produk dengan omzet kecil."""
    return run_query(
        f"""
        SELECT
            products.name AS produk,
            products.category AS kategori,
            COUNT(*) AS jumlah_transaksi,
            SUM(sales.quantity) AS unit_terjual,
            SUM(sales.total) AS omzet
        {JOIN_SQL}
        {where_sql}
        GROUP BY products.id, products.name, products.category
        HAVING SUM(sales.total) >= ?
        ORDER BY omzet DESC, produk
        """,
        [*params, min_revenue],
    )

```

```

def get_report_by_category(where_sql, params):
    """Laporan per kategori produk."""
    return run_query(
        f"""
        SELECT
            products.category AS kategori,
            COUNT(*) AS jumlah_transaksi,
            SUM(sales.quantity) AS unit_terjual,
            SUM(sales.total) AS omzet
        {JOIN_SQL}
        {where_sql}
        GROUP BY products.category
        ORDER BY omzet DESC
        """,
        params,
    )

```

```

def get_report_by_month(where_sql, params):
    """Laporan per bulan; strftime mengambil tahun-bulan dari tanggal."""
    return run_query(
        f"""
        SELECT
            strftime('%Y-%m', sales.sale_date) AS bulan,
            COUNT(*) AS jumlah_transaksi,
            SUM(sales.quantity) AS unit_terjual,
            SUM(sales.total) AS omzet
        {JOIN_SQL}
        {where_sql}
        GROUP BY bulan
        ORDER BY bulan
        """,
        params,
    )

```

```

        """
        ,
        params,
    )

def get_sales_detail(where_sql, params):
    """Data detail (baris per transaksi) untuk menelusuri angka laporan."""
    return run_query(
        f"""
        SELECT
            sales.id AS nomor,
            sales.sale_date AS tanggal,
            products.name AS produk,
            products.category AS kategori,
            sales.unit_price AS harga_satuan,
            sales.quantity AS jumlah,
            sales.total AS total
        {JOIN_SQL}
        {where_sql}
        ORDER BY sales.sale_date, sales.id
        """
        ,
        params,
    )

def format_rupiah(value):
    """Ubah angka menjadi teks rupiah, misalnya 1500000 -> Rp1.500.000."""
    return f"Rp{round(value):,}".replace(",", ".")

def to_csv_bytes(table):
    """Ubah DataFrame menjadi isi file CSV yang rapi saat dibuka di Excel."""
    return table.to_csv(index=False, sep=";").encode("utf-8-sig")

COLUMN_CONFIG = {
    "nomor": st.column_config.NumberColumn("No."),
    "tanggal": st.column_config.TextColumn("Tanggal"),
    "bulan": st.column_config.TextColumn("Bulan"),
    "produk": st.column_config.TextColumn("Produk"),
    "kategori": st.column_config.TextColumn("Kategori"),
    "jumlah_transaksi": st.column_config.NumberColumn("Jumlah transaksi"),
    "unit_terjual": st.column_config.NumberColumn("Unit terjual"),
    "jumlah": st.column_config.NumberColumn("Jumlah unit"),
    "harga_satuan": st.column_config.NumberColumn(
        "Harga satuan (Rp)", format="localized"
    ),
    "total": st.column_config.NumberColumn("Total (Rp)", format="localized"),
    "omzet": st.column_config.NumberColumn("Omzet (Rp)", format="localized"),
}

def show_report(table, file_stem, start_date, end_date):
    """Tampilkan tabel laporan beserta tombol unduh CSV-nya."""
    if table.empty:
        st.info("Tidak ada baris laporan untuk filter ini.")
        return False

    st.dataframe(
        table,
        column_config=COLUMN_CONFIG,
        hide_index=True,
        width="stretch",
    )
    st.download_button(
        "Unduh CSV",
        data=to_csv_bytes(table),
        file_name=f"{file_stem}_{start_date}_{end_date}.csv",
        mime="text/csv",
    )

```

```

        key=f"download_{file_stem}",
    )
    return True

def main():
    st.set_page_config(page_title="Kedai Rasa – Minggu 6", layout="wide")
    st.title("Laporan Penjualan Kedai Rasa")
    st.caption("Latihan Minggu 6. Semua angka berasal dari data simulasi.")

    initialize_database()
    first_date, last_date = get_date_bounds()
    categories = get_categories()

    with st.sidebar:
        st.header("Filter laporan")
        period = st.date_input(
            "Periode",
            value=(first_date, last_date),
            format="DD/MM/YYYY",
        )
        selected_categories = st.multiselect(
            "Kategori", options=categories, default=categories
        )
        min_revenue = st.slider(
            "Omzet minimum per produk (Rp)",
            min_value=0,
            max_value=500000,
            value=0,
            step=25000,
        )

    if len(period) != 2:
        st.info("Pilih tanggal awal dan tanggal akhir periode di sidebar.")
        st.stop()
    if not selected_categories:
        st.warning("Pilih minimal satu kategori di sidebar.")
        st.stop()

    start_date, end_date = period
    where_sql, params = build_filter(start_date, end_date, selected_categories)
    summary = get_summary(where_sql, params)

    st.subheader(f"Ringkasan {start_date:%d/%m/%Y} – {end_date:%d/%m/%Y}")
    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Omzet", format_rupiah(summary["omzet"]))
    col2.metric("Jumlah transaksi", int(summary["jumlah_transaksi"]))
    col3.metric("Rata-rata per transaksi", format_rupiah(summary["rata_rata"]))
    col4.metric("Transaksi terbesar", format_rupiah(summary["transaksi_terbesar"]))

    if summary["jumlah_transaksi"] == 0:
        st.info("Tidak ada transaksi pada periode dan kategori ini.")
        st.stop()

    by_product = get_report_by_product(where_sql, params, min_revenue)
    by_category = get_report_by_category(where_sql, params)
    by_month = get_report_by_month(where_sql, params)
    detail = get_sales_detail(where_sql, params)

    tab_product, tab_category, tab_month, tab_detail = st.tabs(
        ["Per Produk", "Per Kategori", "Per Bulan", "Detail Transaksi"]
    )

    with tab_product:
        st.caption(f"Produk dengan omzet minimal {format_rupiah(min_revenue)}.")
        if show_report(by_product, "laporan-produk", start_date, end_date):
            st.bar_chart(
                by_product,
                x="produk",
            )

```

```

        y="omzet",
        x_label="Produk",
        y_label="Omzet (Rp)",
        sort="-omzet",
        horizontal=True,
    )

with tab_category:
    if show_report(by_category, "laporan-kategori", start_date, end_date):
        st.bar_chart(
            by_category,
            x="kategori",
            y="omzet",
            x_label="Kategori",
            y_label="Omzet (Rp)",
            sort="-omzet",
            horizontal=True,
        )

with tab_month:
    if show_report(by_month, "laporan-bulanan", start_date, end_date):
        st.line_chart(
            by_month, x="bulan", y="omzet", x_label="Bulan", y_label="Omzet (Rp)"
        )

with tab_detail:
    show_report(detail, "detail-transaksi", start_date, end_date)

with st.expander("Cek kecocokan angka"):
    checks = {
        "Kartu Omzet": summary["omzet"],
        "Total tab Per Kategori": by_category["omzet"].sum(),
        "Total tab Per Bulan": by_month["omzet"].sum(),
        "Total tab Detail": detail["total"].sum(),
    }
    for label, value in checks.items():
        st.write(f"{label}: {format_rupiah(value)}")
    if len(set(checks.values())) == 1:
        st.success("Semua total cocok. Laporan dapat ditelusuri ke data detail.")
    else:
        st.error("Ada total yang berbeda. Periksa query laporan.")
    st.caption(
        "Total tab Per Produk bisa lebih kecil jika omzet minimum di atas 0, "
        f"karena HAVING menyaring produk. Saat ini: "
        f"{format_rupiah(by_product['omzet'].sum())}."
    )

if __name__ == "__main__":
    main()

```

3. Jalankan dan amati hasil

Jika Streamlit masih berjalan, klik **Rerun** di pojok kanan atas browser. Jika tidak, jalankan lagi:

```
python -m streamlit run app.py
```

Periksa:

1. Ada empat tab: **Per Produk**, **Per Kategori**, **Per Bulan**, dan **Detail Transaksi**.
2. Setiap tab berisi tabel, tombol **Unduh CSV**, dan grafik (kecuali tab Detail).
3. Panel **Cek kecocokan angka** di bawah tab menampilkan pesan hijau "Semua total cocok".

4. Bedah kode tahap 6.2

a. Laporan per kategori dan per bulan

Per kategori sama dengan per produk, tetapi kelompoknya `products.category`. Per bulan memakai `strftime`:

```
SELECT strftime('%Y-%m', sales.sale_date) AS bulan, ...
GROUP BY bulan
ORDER BY bulan
```

`strftime('%Y-%m', '2026-09-14')` menghasilkan `'2026-09'`. Semua transaksi September berubah menjadi teks yang sama, sehingga `GROUP BY bulan` mengumpulkannya dalam satu kelompok. Urutan `ORDER BY bulan` naik (lama ke baru) karena laporan waktu dibaca dari kiri ke kanan.

b. Satu fungsi untuk menampilkan semua laporan

```
def show_report(table, file_stem, start_date, end_date):
    if table.empty:
        st.info("Tidak ada baris laporan untuk filter ini.")
        return False
    st.dataframe(table, column_config=COLUMN_CONFIG, hide_index=True, width="stretch")
    st.download_button(
        "Unduh CSV",
        data=to_csv_bytes(table),
        file_name=f"{file_stem}_{start_date}_{end_date}.csv",
        mime="text/csv",
        key=f"download_{file_stem}",
    )
    return True
```

Keempat tab memanggil fungsi yang sama, jadi tampilan, nama file, dan penanganan data kosong seragam. Fungsi mengembalikan `True` jika tabel tampil, dan grafik hanya dibuat jika ada data. `COLUMN_CONFIG` berisi semua kolom yang mungkin muncul; Streamlit hanya memakai yang ada di tabel.

`key` memberi identitas unik untuk setiap tombol. Streamlit membedakan tombol dari label dan isinya. Jika dua tombol kebetulan berlabel dan berisi sama, Streamlit menampilkan error *StreamlitDuplicateElementId*. Karena keempat tombol berlabel "Unduh CSV", memberi `key` unik adalah kebiasaan yang aman.

c. `st.tabs`

```
tab_product, tab_category, tab_month, tab_detail = st.tabs(
    ["Per Produk", "Per Kategori", "Per Bulan", "Detail Transaksi"]
)
with tab_product:
    ...
```

Tab hanya mengatur **tampilan**. Semua isi tab tetap dihitung setiap kali skrip berjalan, sehingga keempat laporan selalu konsisten dengan filter yang sama.

d. Grafik sebagai pelengkap

```
st.bar_chart(by_product, x="produk", y="omzet",
              x_label="Produk", y_label="Omzet (Rp)",
              sort="-omzet", horizontal=True)
```

- `sort="-omzet"` mengurutkan batang dari omzet terbesar (tanda `-` berarti menurun). Tanpa opsi ini, batang diurutkan menurut abjad nama.
- `horizontal=True` membuat batang mendatar sehingga nama produk terbaca tanpa dimiringkan.
- Tab bulan memakai `st.line_chart` karena data waktu lebih mudah dibaca sebagai garis naik-turun.
- Grafik menunjukkan **pola**, tabel menunjukkan **angka pasti**. Jika keduanya tampak berbeda, percayai tabel lalu periksa grafiknya.

e. Mengubah DataFrame menjadi CSV

```
def to_csv_bytes(table):
    return table.to_csv(index=False, sep=";").encode("utf-8-sig")
```

Bagian	Alasan
<code>index=False</code>	Nomor baris internal pandas (0, 1, 2, ...) tidak ikut ke file.
<code>sep=";"</code>	Excel dengan pengaturan bahasa Indonesia memakai koma sebagai tanda desimal, sehingga memisahkan kolom dengan titik koma. Dengan <code>;</code> , file langsung terbagi menjadi kolom saat dibuka.
<code>.encode("utf-8-sig")</code>	Menambahkan penanda UTF-8 agar huruf dan simbol seperti <code>"–"</code> tidak berubah menjadi karakter aneh di Excel.

Jika file akan diolah dengan Google Sheets atau program lain yang memakai koma, ganti `sep=";"` menjadi `sep=","`. Yang penting, pemisah yang dipakai **disepakati dan dicatat**.

Angka di CSV disimpan **tanpa** pemisah ribuan (misalnya `513000`). Dengan begitu spreadsheet membacanya sebagai angka yang bisa dijumlahkan, bukan teks.

f. Tombol unduh dan nama file

`st.download_button` mengirim isi `data` ke browser sebagai file. Nama file memuat jenis laporan dan periodenya, misalnya:

```
laporan-produk_2026-08-03_2026-09-30.csv
laporan-kategori_2026-09-01_2026-09-30.csv
```

Nama seperti ini membuat file mudah dicari dan tidak tertukar antarperiode. Kolom dan urutannya sama persis dengan tabel di layar, karena keduanya berasal dari DataFrame yang sama.

g. Panel cek kecocokan angka (rekonsiliasi)

```
checks = {
    "Kartu Omzet": summary["omzet"],
    "Total tab Per Kategori": by_category["omzet"].sum(),
    "Total tab Per Bulan": by_month["omzet"].sum(),
    "Total tab Detail": detail["total"].sum(),
}
if len(set(checks.values())) == 1:
    st.success("Semua total cocok. ...")
```

Keempat angka dihitung dengan cara berbeda, tetapi harus sama karena berasal dari baris yang sama.

`set(...)` membuang nilai kembar; jika hanya tersisa satu nilai, semua total cocok. Jika suatu hari ada yang mengubah query (misalnya lupa memasang `WHERE` di salah satu laporan), panel ini langsung menampilkan

pesan merah.

Total tab **Per Produk** sengaja tidak dimasukkan ke pengecekan karena **HAVING** bisa membuang produk. Saat slider = 0, totalnya juga harus sama.

5. Buka file CSV

1. Di tab **Per Kategori**, klik **Unduh CSV**. File tersimpan di folder *Downloads*.
2. Buka file dengan Excel (atau impor ke Google Sheets dengan pemisah titik koma).
3. Pastikan isinya tiga baris kategori dengan kolom `kategori`, `jumlah_transaksi`, `unit_terjual`, dan `omzet`.
4. Di spreadsheet, jumlahkan kolom `omzet`, misalnya `=SUM(D2:D4)`. Hasilnya harus sama dengan kartu **Omzet**.
5. Ulangi dengan tab **Detail Transaksi**, lalu jumlahkan kolom `total`.

Contoh isi `laporan-kategori_2026-08-03_2026-09-30.csv` jika dibuka di Notepad:

```
kategori;jumlah_transaksi;unit_terjual;omzet
Minuman;16;62;905000
Makanan;10;26;440000
Camilan;4;18;144000
```

6. Uji di aplikasi

Kasus	Tindakan	Hasil yang diharapkan
Rekonsiliasi seluruh periode	Filter bawaan, buka Cek kecocokan angka	Empat total = Rp1.489.000; pesan hijau.
Rekonsiliasi dengan filter	Periode September, kategori Minuman	Empat total = Rp485.000; pesan hijau.
Laporan per bulan	Filter bawaan	2026-08 = 690.000; 2026-09 = 799.000.
CSV sesuai tampilan	Unduh tab Per Kategori dan jumlahkan <code>omzet</code> di spreadsheet	Jumlah = kartu Omzet; tiga baris kategori.
Nama file mengikuti filter	Ubah periode lalu unduh lagi	Tanggal di nama file ikut berubah.
HAVING dan CSV	Slider Rp200.000, unduh tab Per Produk	CSV hanya berisi tiga produk, sama dengan tabel di layar.
Periode kosong	Periode Oktober 2026	Kartu Rp0; tab dan tombol unduh tidak tampil; tidak error.

7. Checkpoint pertemuan 6.2

- [] Empat tab laporan tampil dan mengikuti filter yang sama.
- [] Setiap tab memiliki tombol **Unduh CSV** dengan nama file yang memuat periode.
- [] File CSV terbuka rapi di spreadsheet dan jumlahnya sama dengan kartu Omzet.
- [] Grafik tampil sebagai pelengkap, bukan pengganti tabel.
- [] Panel **Cek kecocokan angka** menunjukkan semua total cocok.

- [] Database minggu sebelumnya tidak diubah.

H. Panduan troubleshooting

Gejala	Kemungkinan penyebab	Solusi
<code>ModuleNotFoundError: No module named 'pandas'</code>	Streamlit belum terpasang di environment yang aktif.	Aktifkan <code>.venv</code> , lalu <code>python -m pip install --upgrade streamlit</code> .
<code>TypeError: ... unexpected keyword argument 'sort'</code>	Versi Streamlit lama.	<code>python -m pip install --upgrade streamlit</code> , lalu jalankan ulang.
Angka berbeda dari angka acuan	Data simulasi pernah diubah, atau database lama ikut tersalin.	Tutup Streamlit, hapus <code>penjualan_m6.db</code> di folder <code>pdb-minggu-06</code> , lalu jalankan lagi.
<code>StreamlitDuplicateElementId</code>	Dua tombol unduh berlabel dan berisi sama, tanpa <code>key</code> berbeda.	Pastikan setiap <code>st.download_button</code> memiliki <code>key</code> unik.
<code>DatabaseError: Execution failed on sql ... Incorrect number of bindings supplied</code>	Jumlah <code>?</code> tidak sama dengan jumlah nilai di <code>params</code> .	Periksa <code>build_filter; HAVING ... ?</code> membutuhkan satu nilai tambahan.
<code>DatabaseError: ... misuse of aggregate function sum()</code>	<code>SUM(...)</code> ditulis di <code>WHERE</code> .	Pindahkan syarat agregat ke <code>HAVING</code> .
Kolom CSV menumpuk di kolom A Excel	Pemisah tidak cocok dengan pengaturan Excel.	Pakai Data → From Text/CSV dan pilih pemisah <code>;</code> , atau ganti <code>sep</code> .
Huruf aneh seperti <code>â€œ</code> di Excel	File disimpan tanpa penanda UTF-8.	Pastikan memakai <code>.encode("utf-8-sig")</code> .
Kartu menampilkan <code>nan</code> atau error saat periode kosong	<code>COALESCE</code> terhapus.	Kembalikan <code>COALESCE(..., 0)</code> pada <code>SUM</code> , <code>AVG</code> , dan <code>MAX</code> .

I. Pengumpulan dan refleksi

Kumpulkan dalam satu folder/arsip:

- `app.py` versi akhir Pertemuan 6.2;
- dua file CSV hasil unduhan (misalnya Per Kategori dan Per Produk) untuk periode yang dicatat;
- `catatan-pengujian.txt` berisi hasil uji 6.1 dan 6.2, termasuk minimal satu rekonsiliasi angka;
- satu tangkapan layar halaman laporan dengan panel **Cek kecocokan angka** terbuka.

Jangan mengumpulkan `.venv`, kata sandi, token, atau database berisi data nyata.

Pertanyaan refleksi

1. Apa perbedaan `WHERE` dan `HAVING`? Beri contoh dari aplikasi.
2. Mengapa nilai filter dikirim lewat parameter `?` dan bukan ditempel ke teks SQL?

3. Apa yang terjadi pada `SUM` dan `AVG` jika tidak ada baris yang lolos filter, dan bagaimana aplikasi mengatasinya?
4. Pisang Goreng terjual lebih banyak unit daripada Nasi Goreng tetapi omzetnya lebih kecil. Laporan mana yang Anda tunjukkan kepada pemilik usaha untuk keputusan stok, dan mana untuk keputusan harga?
5. Mengapa laporan perlu dapat diekspor ke CSV, dan mengapa angkanya perlu bisa ditelusuri ke data detail?
6. Mengapa angka di CSV disimpan tanpa pemisah ribuan?

Rubrik umpan balik formatif

Aspek	Bukti yang diamati
Ketepatan angka	Kartu dan laporan cocok dengan angka acuan dan hitung manual.
Filter dan parameter	Periode/kategori dikirim lewat <code>?</code> ; semua laporan berubah bersama.
Agregasi	<code>GROUP BY</code> , <code>HAVING</code> , dan <code>ORDER BY</code> dipakai sesuai tujuan.
Data kosong	Periode kosong, kategori kosong, dan <code>HAVING</code> tinggi tidak menimbulkan error.
Penyajian	Tab, judul kolom, format rupiah, dan grafik rapi serta mudah dibaca.
Ekspor dan rekonsiliasi	CSV sesuai tampilan; total antar-laporan terbukti cocok.

Rubrik ini panduan umpan balik, bukan bobot nilai resmi.

J. Ringkasan

Minggu 3 menyimpan data, Minggu 4 mencari dan meringkas, Minggu 5 menjaga mutu dan relasi data. Minggu 6 mengubah data transaksi menjadi **laporan yang bisa dipakai untuk keputusan**:

- `WHERE` memilih baris, `GROUP BY` mengelompokkan, fungsi agregasi menghitung, `HAVING` menyaring kelompok, dan `ORDER BY` mengurutkan;
- filter Streamlit dikirim ke query lewat parameter, sehingga semua angka konsisten dan aman;
- tab, format kolom, dan grafik membuat laporan mudah dibaca;
- CSV membuat laporan bisa dibagikan, dan rekonsiliasi membuktikan angkanya dapat dipercaya.

Halaman laporan ini dapat dipakai sebagai bukti kemajuan proyek menjelang UTS.